TD : connecter l'interface à l'API sécurisée

Durée indicative : 65 minutes. Vous partez de l'interface du jour 2 et vous allez ajouter la connexion, la liste personnelle et l'envoi et le téléchargement d'un PDF privé. La branche `jour4-interface-api-securisee` contient la correction.

1. Préparer les deux projets — 5 min

Ouvrez les deux dossiers dans VS Code. Dans le terminal de **chaque** projet, vérifiez votre branche et vos fichiers :

```
git branch --show-current
git status
```

- **Interface** : utilisez `jour2-fetch-documents`, qui contient ce guide. Sauvegardez votre travail avant de changer de branche. Si Git refuse, demandez au formateur ; ne supprimez pas vos fichiers.
- **Laravel** : utilisez votre API sécurisée terminée ou la branche `jour3-securite-documents`. Elle doit déjà être configurée, avec ses migrations et deux comptes fictifs préparés pendant le jour 3. Démarrez les services habituels si nécessaire depuis le dossier Laravel avec `docker compose up -d`.
- Utilisez uniquement la **base locale de formation** et des fichiers fictifs. Ce TD ne demande aucune migration ni remise à zéro. Si les comptes ne sont pas prêts, demandez au formateur de vérifier la base et de reprendre le guide de sécurité du jour 3.

À vérifier : dans Postman, `GET http://localhost:8088/api/documents` avec `Accept: application/json` et sans token répond **401**. Si l'API ne répond pas, vérifiez Docker et la branche Laravel avant de poursuivre. Les comptes du TD précédent sont `amina@example.test` et `karim@example.test`, avec le mot de passe de formation `Formation123!`, sauf si le formateur vous en a donné d'autres. Un compte préexistant peut avoir un autre mot de passe.

Ouvrez `index.html` dans votre navigateur. À chaque étape, enregistrez avec **Ctrl+S** et rechargez la page. Le token de l'exercice reste uniquement dans la mémoire de la page : après un rechargement, il faut se reconnecter. Le rechargement n'appelle pas la route de révocation : utilisez **Se déconnecter** lorsque c'est possible.

2. Ajouter le formulaire de connexion — 10 min

Dans `index.html`, cherchez cette ligne :

```
    <div class="row g-4 align-items-start">
```

Ajoutez **juste avant** cette ligne le bloc :

```
    <section class="card app-card mb-4" aria-labelledby="login-title">
      <div class="card-body p-4">
        <h2 id="login-title" class="section-title">Connexion</h2>
        <p class="section-subtitle">Connectez-vous avec un compte de formation.</p>
        <p id="login-message" role="status" aria-live="polite"></p>
        <form id="login-form" class="row g-3 align-items-end">
```

```
          <div class="col-md-5">
            <label class="form-label" for="login-email">E-mail</label>
            <input class="form-control" id="login-email" type="email" autocomplete="username" required>
          </div>
          <div class="col-md-5">
            <label class="form-label" for="login-password">Mot de passe</label>
            <input class="form-control" id="login-password" type="password" autocomplete="current-password" required>
          </div>
          <div class="col-md-2"><button class="btn btn-primary w-100" type="submit">Se connecter</button></div>
        </form>
        <button id="logout-button" type="button" class="btn btn-outline-secondary d-none">Se déconnecter</button>
      </div>
    </section>
```

Remplacez ensuite la ligne `<div class="row g-4 align-items-start">` par :

```
    <div id="connected-content" class="row g-4 align-items-start d-none">
```

Dans le formulaire d'ajout, remplacez le texte « Seuls le titre et la catégorie sont enregistrés. L'envoi de fichiers sera étudié plus tard. » par « Après la création, vous pourrez ajouter un PDF depuis la liste. »

À vérifier : rechargez la page. Le formulaire de connexion apparaît et les zones de documents sont masquées. À ce stade, l'ancien JavaScript ne gère pas encore le nouveau formulaire : c'est normal.

3. Connecter et déconnecter — 15 min

Dans `assets/js/app.js`, **remplacez tout le contenu** par le code ci-dessous. Il conserve le token dans une variable, ajoute l'en-tête Bearer aux requêtes protégées et efface les données affichées à la déconnexion.

```
// La préférence d'affichage est gérée séparément dans securite.js.
const loginForm = document.querySelector('#login-form');
const loginMessage = document.querySelector('#login-message');
const logoutButton = document.querySelector('#logout-button');
const connectedContent = document.querySelector('#connected-content');
const form = document.querySelector('#document-form');
const titleInput = document.querySelector('#document-title');
const categoryInput = document.querySelector('#document-category');
const documentList = document.querySelector('#document-list');
const documentCount = document.querySelector('#document-count');
const listMessage = document.querySelector('#list-message');
const formMessage = document.querySelector('#form-message');
const submitButton = form.querySelector('button[type="submit"]');
const apiUrl = 'http://localhost:8088/api';
let token = null; // Conservé uniquement en mémoire, jusqu'au rechargement de la page.

function showFormMessage(message, success) {
  formMessage.textContent = message;
  formMessage.classList.remove('d-none', 'alert-success', 'alert-danger');
  formMessage.classList.add(success ? 'alert-success' : 'alert-danger');
}

function closeSession(message) {
  token = null;
  documentList.replaceChildren();
  documentCount.textContent = '0 document';
  listMessage.textContent = '';
  form.reset();
  form.classList.remove('was-validated');
  titleInput.setCustomValidity('');
  formMessage.classList.add('d-none');
```

```
  connectedContent.classList.add('d-none');
  logoutButton.classList.add('d-none');
  loginForm.classList.remove('d-none');
  loginMessage.textContent = message;
}

// Chaque demande protégée reçoit le token de la session qui l'a démarrée.
async function authenticatedFetch(path, options = {}, sessionToken = token) {
  const headers = new Headers(options.headers || {});
  headers.set('Accept', 'application/json');
  headers.set('Authorization', `Bearer ${sessionToken}`);
  const response = await fetch(`${apiUrl}${path}`, { ...options, headers });
  if (response.status === 401 && token === sessionToken) {
    closeSession('Session expirée ou révoquée. Connectez-vous à nouveau.');
  }
  return response;
}

loginForm.addEventListener('submit', async (event) => {
  event.preventDefault();
  const button = loginForm.querySelector('button');
  button.disabled = true;
  loginMessage.textContent = 'Connexion en cours…';
  try {
    const response = await fetch(`${apiUrl}/login`, {
      method: 'POST',
      headers: { Accept: 'application/json', 'Content-Type': 'application/json' },
      body: JSON.stringify({
        email: document.querySelector('#login-email').value.trim(),
        password: document.querySelector('#login-password').value,
      }),
    });
    if (response.status === 401 || response.status === 422) throw new Error('Vérifiez votre e-mail et votre mot de
passe.');
    if (response.status === 429) throw new Error('Trop de tentatives. Patientez une minute.');
    if (!response.ok) throw new Error('Connexion impossible. Vérifiez que Laravel fonctionne.');
    const data = await response.json();
    token = data.token;
    document.querySelector('#login-password').value = '';
    loginForm.classList.add('d-none');
    logoutButton.classList.remove('d-none');
    connectedContent.classList.remove('d-none');
    loginMessage.textContent = 'Vous êtes connecté. Le token reste en mémoire dans cette page.';
    await loadDocuments(token);
  } catch (error) {
    loginMessage.textContent = error instanceof TypeError ? 'Serveur inaccessible. Vérifiez que Laravel fonctionne.' :
error.message;
  } finally {
    button.disabled = false;
  }
});

logoutButton.addEventListener('click', async () => {
  const sessionToken = token;
  logoutButton.disabled = true;
  try {
    const response = await authenticatedFetch('/logout', { method: 'POST' }, sessionToken);
    if (token !== sessionToken) return;
    closeSession(response.ok ? 'Vous êtes déconnecté.' : 'Déconnexion locale effectuée. La révocation du token n’a pas
été confirmée.');
  } catch (error) {
    if (token === sessionToken) closeSession('Déconnexion locale effectuée. La révocation du token n’a pas été
confirmée.');
  } finally {
    logoutButton.disabled = false;
  }
});
```

```
// Fonction provisoire : la liste complète sera ajoutée à l'étape suivante.
async function loadDocuments() {
  listMessage.textContent = 'Connexion réussie : la liste sera ajoutée à l’étape suivante.';
}
```

À vérifier : enregistrez puis rechargez. Essayez un mot de passe incorrect : un message apparaît. Connectez Amina : les zones de documents apparaissent et la liste indique que sa construction vient ensuite. Cliquez sur **Se déconnecter** : les zones disparaissent. Si une erreur 429 apparaît, attendez une minute : la route de connexion est déjà limitée dans l’API du jour 3.

Le bouton de déconnexion demande à Laravel de révoquer le token. Si le serveur est inaccessible, l’interface efface quand même la session locale et indique que la révocation n’a pas été confirmée.

4. Afficher les documents et gérer les PDF — 20 min

Dans `app.js`, repérez et **remplacez uniquement** le commentaire et la fonction provisoire ci-dessous, sans retirer le reste du fichier :

```
// Fonction provisoire : la liste complète sera ajoutée à l'étape suivante.
async function loadDocuments() {
  listMessage.textContent = 'Connexion réussie : la liste sera ajoutée à l’étape suivante.';
}
```

Collez à sa place ce code complet :

```
function createDocument(documentData, sessionToken) {
  const article = document.createElement('article');
  article.className = 'document-item flex-wrap';
  const details = document.createElement('div');
  details.className = 'document-details';
  const heading = document.createElement('h3');
  heading.textContent = documentData.title;
  const metadata = document.createElement('p');
  metadata.textContent = documentData.category;
  const status = document.createElement('p');
  status.setAttribute('role', 'status');
  status.setAttribute('aria-live', 'polite');
  const fileInput = document.createElement('input');
  fileInput.type = 'file';
  fileInput.accept = '.pdf,application/pdf';
  fileInput.className = 'form-control form-control-sm mt-2';
  fileInput.setAttribute('aria-label', `Choisir un PDF pour ${documentData.title}`);
  const upload = document.createElement('button');
  upload.type = 'button';
  upload.className = 'btn btn-sm btn-outline-primary mt-2 me-2';
  upload.textContent = 'Envoyer le PDF';
  const download = document.createElement('button');
  download.type = 'button';
  download.className = 'btn btn-sm btn-outline-secondary mt-2';
  download.textContent = 'Télécharger le PDF';
  details.append(heading, metadata, fileInput, upload, download, status);
  article.append(details);

  upload.addEventListener('click', async () => {
    const file = fileInput.files[0];
    if (!file) { status.textContent = 'Choisissez un PDF avant l’envoi.'; return; }
    const data = new FormData();
    data.append('file', file);
    upload.disabled = true;
    status.textContent = 'Envoi en cours…';
```

```
    try {
      const response = await authenticatedFetch(`/documents/${documentData.id}/file`, { method: 'POST', body: data },
sessionToken);
      if (token !== sessionToken) return;
      if (response.status === 422) throw new Error('Fichier refusé : choisissez un vrai PDF de 1 Mo maximum.');
      if (response.status === 409) throw new Error('Ce document possède déjà un PDF.');
      if (!response.ok) throw new Error('Envoi impossible. Vérifiez votre accès et réessayez.');
      fileInput.value = '';
      status.textContent = 'PDF enregistré.';
    } catch (error) {
      if (token === sessionToken) status.textContent = error instanceof TypeError ? 'Serveur inaccessible. Réessayez.' :
error.message;
    } finally { upload.disabled = false; }
  });

  download.addEventListener('click', async () => {
    download.disabled = true;
    status.textContent = 'Téléchargement en cours…';
    try {
      const response = await authenticatedFetch(`/documents/${documentData.id}/file`, {}, sessionToken);
      if (token !== sessionToken) return;
      if (response.status === 404) throw new Error('Aucun PDF accessible pour ce document.');
      if (!response.ok) throw new Error('Téléchargement impossible. Réessayez.');
      const blob = await response.blob();
      if (token !== sessionToken) return;
      const url = URL.createObjectURL(blob);
      const link = document.createElement('a');
      link.href = url;
      link.download = `document-${documentData.id}.pdf`;
      document.body.append(link);
      link.click();
      link.remove();
      setTimeout(() => URL.revokeObjectURL(url), 1000);
      status.textContent = 'Téléchargement lancé.';
    } catch (error) {
      if (token === sessionToken) status.textContent = error instanceof TypeError ? 'Serveur inaccessible. Réessayez.' :
error.message;
    } finally { download.disabled = false; }
  });
  return article;
}

async function loadDocuments(sessionToken) {
  listMessage.textContent = 'Chargement des documents…';
  try {
    const response = await authenticatedFetch('/documents', {}, sessionToken);
    if (token !== sessionToken) return;
    if (!response.ok) throw new Error('Liste indisponible.');
    const documents = await response.json();
    if (token !== sessionToken) return;
    documentList.replaceChildren();
    documents.forEach((item) => documentList.append(createDocument(item, sessionToken)));
    documentCount.textContent = `${documents.length} document${documents.length > 1 ? 's' : ''}`;
    listMessage.textContent = documents.length ? '' : 'Aucun document pour le moment.';
  } catch (error) {
    if (token !== sessionToken) return;
    documentList.replaceChildren();
    documentCount.textContent = 'Indisponible';
    listMessage.textContent = 'Impossible de charger la liste. Vérifiez Laravel puis reconnectez-vous.';
  }
}
```

`loadDocuments` utilise le token pour demander la liste personnelle. Pour chaque document, `createDocument` construit une ligne avec `textContent`, ce qui affiche les valeurs comme du texte. Le fichier est envoyé avec `FormData` sous la clé `file` ; ne fixez pas manuellement son `Content-Type`. Le téléchargement utilise `fetch`, puis un `Blob`, pour envoyer le même token à Laravel.

À vérifier : rechargez, reconnectez Amina et observez sa liste. Elle peut être vide : les anciens documents sans propriétaire ne sont attribués à personne. Si un document personnel existe déjà, ses boutons **Envoyer le PDF** et **Télécharger le PDF** apparaissent. Nous ajouterons un nouveau document à l'étape suivante.

5. Ajouter un document et vérifier le parcours — 15 min

Dans `app.js`, allez **tout à la fin** du fichier et ajoutez :

```
form.addEventListener('submit', async (event) => {
  event.preventDefault();
  const sessionToken = token;
  const title = titleInput.value.trim();
  titleInput.setCustomValidity(title.length < 3 ? 'Saisissez au moins 3 caractères.' : '');
  form.classList.add('was-validated');
  formMessage.classList.add('d-none');
  if (!form.checkValidity()) { form.querySelector(':invalid')?.focus(); return; }
  submitButton.disabled = true;
  try {
    const response = await authenticatedFetch('/documents', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title, category: categoryInput.value }),
    }, sessionToken);
    if (token !== sessionToken) return;
    if (response.status === 422) throw new Error('Le titre ou la catégorie est invalide.');
    if (!response.ok) throw new Error('Impossible d'ajouter le document. Réessayez.');
    form.reset();
    form.classList.remove('was-validated');
    titleInput.setCustomValidity('');
    showFormMessage('Document enregistré. Vous pouvez lui ajouter un PDF dans la liste.', true);
    await loadDocuments(sessionToken);
  } catch (error) {
    if (token === sessionToken) showFormMessage(error instanceof TypeError ? 'Serveur inaccessible. Réessayez.' :
error.message, false);
  } finally { submitButton.disabled = false; }
});

titleInput.addEventListener('input', () => titleInput.setCustomValidity(''));
document.querySelectorAll('#navigation .nav-link').forEach((link) => {
  link.addEventListener('click', () => {
    const navigation = document.querySelector('#navigation');
    if (navigation.classList.contains('show')) bootstrap.Collapse.getOrCreateInstance(navigation).hide();
  });
});
```

1. Enregistrez et rechargez, puis connectez Amina. Créez un document avec un titre d'au moins trois caractères et une catégorie. **À vérifier :** le document apparaît dans sa liste.
2. Sur sa ligne, choisissez un vrai PDF fictif de **1 Mo maximum**, puis cliquez sur **Envoyer le PDF**. **À vérifier :** le message confirme l'enregistrement. Un second envoi sur ce document reçoit **409** : l'API n'accepte qu'un fichier par document.
3. Cliquez sur **Télécharger le PDF**, puis ouvrez le fichier téléchargé. **À vérifier :** son contenu correspond à l'original. Si aucun fichier n'a été envoyé, l'API répond **404** et la page indique qu'aucun PDF n'est accessible.
4. Créez un autre document et envoyez-lui un fichier texte renommé `faux.pdf`. **À vérifier :** Laravel le refuse avec **422**. Le titre reste enregistré : choisissez ensuite un vrai PDF sur la même ligne.

5. Déconnectez Amina et connectez Karim. **À vérifier :** le document d'Amina ne figure pas dans sa liste. Avec Postman et le token de Karim, essayez `GET /api/documents/{id}/file` pour l'identifiant d'Amina : la réponse est **404**. N'utilisez pas le token d'Amina pour cet essai.
6. Déconnectez Karim. **À vérifier :** l'interface masque les documents. Rechargez la page : une nouvelle connexion est nécessaire.

La création du document et l'envoi du fichier sont deux opérations différentes. Si l'envoi échoue, le document existe déjà : corrigez le fichier et recommencez depuis sa ligne, sans recréer le document.

Si une étape ne fonctionne pas

Situation	Vérification
Serveur inaccessible	Vérifiez Docker, la branche Laravel et `http://localhost:8088` avec le formateur.
401 ou retour au formulaire de connexion	Le token a expiré ou a été révoqué : reconnectez-vous.
Liste vide	Vérifiez le compte. Les anciens documents sans propriétaire restent invisibles.
422 au téléversement	Choisissez un vrai PDF de 1 Mo maximum. Le nom seul ne suffit pas.
409 au téléversement	Un PDF est déjà associé à ce document.
Téléchargement 404	Vérifiez le compte, le document et la présence d'un fichier.
Erreur CORS dans la console	Vérifiez l'adresse API et la configuration locale Laravel avec le formateur.

Comparer avec la correction

La branche `jour4-interface-api-securisee` contient la correction. Sauvegardez vos fichiers avant tout changement de branche. Pour comparer les fichiers suivis sans changer de branche, après récupération éventuelle de la branche de correction :

```
git diff jour4-interface-api-securisee -- index.html assets/js/app.js
```

Discutez avec le formateur : pourquoi le token est-il envoyé à chaque demande protégée ? Pourquoi le contrôle du propriétaire et la validation du PDF restent-ils nécessaires côté serveur ?